Automatización de Modelado para STAAD.Pro

Autor: Jose Enrique ... (Completar con nombre real)

Año: 2026

Lugar: México

Ficha Técnica y Síntesis

El presente software es una herramienta informática desarrollada con el objetivo principal de automatizar y optimizar el proceso de modelado geométrico y estructural de naves industriales. Su función central consiste en generar de manera automatizada y eficiente los modelos tridimensionales requeridos para su integración y análisis en el software de ingeniería estructural STAAD.Pro, reduciendo significativamente el tiempo de modelado manual y minimizando la posibilidad de errores operativos.

Diagrama de Arquitectura y Flujo

1. Interfaz de Usuario (UI - Frontend):

   - Captura de parámetros estructurales (dimensiones, marcos, largueros, etc.).

   - Validación de datos de entrada.

2. Motor Lógico (Backend):

   - Procesamiento de parámetros y generación de nodos y conectividades.

   - Asignación de propiedades y perfiles estructurales.

3. Integración con STAAD.Pro:

   - Exportación de comandos OpenSTAAD y generación de archivos compatibles (.std).

   - Visualización 3D del modelo resultante.

Muestra de Código - Interfaz de Usuario (Primeras páginas)

```
import streamlit as st
import math
import os
import sys
import subprocess
import importlib

# Silenciar advertencia de valores por defecto en widgets con Session State
try:
    import streamlit.elements.lib.policies as _policies
    _policies._shown_default_value_warning = True
except Exception:
    pass


# Ensure app directory and root directory are in sys.path for both local and web environments
app_dir = os.path.dirname(os.path.abspath(__file__))
root_dir = os.path.dirname(app_dir)
if app_dir not in sys.path:
    sys.path.insert(0, app_dir)
if root_dir not in sys.path:
    sys.path.insert(0, root_dir)

import streamlit.components.v1 as components

# Force reload of custom modules to ensure Streamlit hot-reloads them (Only in Local Dev)
if os.environ.get("LOCAL_DEV") == "1":
    import model
    import staad_writer
    import visualizer
    import prediseno
    importlib.reload(model)
    importlib.reload(staad_writer)
    importlib.reload(visualizer)
    importlib.reload(prediseno)
import model
import staad_writer
import visualizer
import prediseno
try:
    import Catalogo
    import Catalogo.catalogo_ui
    import Catalogo.catalogo_state
    importlib.reload(Catalogo)
    importlib.reload(Catalogo.catalogo_ui)
    importlib.reload(Catalogo.catalogo_state)
except Exception:
    pass


from model import StaadModel
from staad_writer import write_staad
from visualizer import plot_model

# Import configurations
is_mobile_client = False
try:
    import js
    if js.window.innerWidth <= 768:
```

```
        is_mobile_client = True
except Exception:
    pass


# Configuración de icono: "black", "dark_slate" o "blue" (cambiar aquí para alternar color)
ICONO_COLOR = "blue"
icono_filename = {
    "black": "staad_black.ico",
    "dark_slate": "staad_darkslate.ico",
    "blue": "staad_blue.ico"
}.get(ICONO_COLOR, "staad_blue.ico")

icono_path = os.path.join(os.path.dirname(__file__), "..", icono_filename)
if not os.path.exists(icono_path):
    icono_path = os.path.join(os.path.dirname(__file__), "..", "staad_blue.ico")

page_icon = icono_path if os.path.exists(icono_path) else ""
st.set_page_config(page_title="Naves Industriales STAAD.Pro", page_icon=page_icon, layout="wide")

st.markdown("""
<style>
/* Mobile layout adjustments for Plotly 3D Viewer */
@media screen and (max-width: 768px) {
    div[data-testid="stPlotlyChart"],
    div[data-testid="stPlotlyChart"] iframe,
    div[data-testid="stPlotlyChart"] .js-plotly-plot,
    div[data-testid="stPlotlyChart"] .plot-container.plotly,
    div[data-testid="stPlotlyChart"] .svg-container {
        height: 450px !important;
        min-height: 450px !important;
        max-height: 450px !important;
    }
}

/* iOS Liquid Glass Effect for containers */
div[data-testid="stVerticalBlock"]:has(> div.element-container div.ios-glass-marker) {
    background: rgba(255, 255, 255, 0.45);
    backdrop-filter: blur(15px);
    -webkit-backdrop-filter: blur(15px);
    border-radius: 16px;
    padding: 12px 16px;
    box-shadow: 0 4px 30px rgba(0, 0, 0, 0.1);
    border: 1px solid rgba(255, 255, 255, 0.8);
    margin-bottom: 10px;
    position: relative;
    top: -95px;
    z-index: 99;
}
/* Adjust spacing inside the glass card */
div[data-testid="stVerticalBlock"]:has(> div.element-container div.ios-glass-marker) h3 {
    margin-top: -10px;
    font-weight: 600;
}
/* Tamaño de fuente reducido para la Ubicación */
div[data-testid="stVerticalBlock"]:has(> div.element-container div.path-marker) input {
    font-size: 11px !important;
}
/* Ocultar el contenedor del marcador para que no desalinee el input */
div.element-container:has(div.path-marker) {
    display: none !important;
    margin: 0 !important;
    height: 0 !important;
```

```
}
/* Sidebar Logo Styling */
[data-testid="stSidebar"] [data-testid="stImage"] {
    display: flex;
    justify-content: center;
    align-items: center;
    width: 100%;
}
[data-testid="stSidebar"] img {
    border-radius: 18px !important;
    box-shadow: 0 12px 24px rgba(0, 0, 0, 0.25) !important;
    width: 66% !important;
    margin: 0 auto 15px auto !important;
    display: block !important;
}
/* Make the divider faint grey */
hr {
    margin: 15px 0 !important;
    border-top: 1px solid rgba(0,0,0,0.1) !important;
}
/* Reduce overall top padding to avoid scroll */
.block-container {
    padding-top: 1.5rem !important;
    padding-bottom: 1rem !important;
}
</style>
""", unsafe_allow_html=True)


# CSS para botones
st.markdown("""
    <style>
    /* Variables globales de tema */
    :root, html, body, #root, [data-testid="stAppViewContainer"] {
        --primary-color: #005cb2 !important;
        font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif !important;
    }

    /* Reducción general de fuentes para la barra lateral y controles */
    section[data-testid="stSidebar"] h1,
    section[data-testid="stSidebar"] h2 {
        font-size: 18px !important;
        font-weight: 600 !important;
        color: #0f172a !important;
        margin-top: 4px !important;
        margin-bottom: 8px !important;
        letter-spacing: -0.2px !important;
    }

    section[data-testid="stSidebar"] label,
    section[data-testid="stSidebar"] label p,
    section[data-testid="stSidebar"] [data-testid="stWidgetLabel"] p,
    section[data-testid="stSidebar"] [data-testid="stWidgetLabel"] span {
        font-size: 12px !important;
        font-weight: 500 !important;
        color: #334155 !important;
        margin-bottom: 2px !important;
        line-height: 1.25 !important;
    }

    section[data-testid="stSidebar"] div[role="radiogroup"] label p,
    section[data-testid="stSidebar"] [data-testid="stCheckbox"] label p {
        font-size: 12.5px !important;
```

```
    font-weight: normal !important;
    color: #1e293b !important;
}

/* Inputs numéricos y textos en sidebar */
section[data-testid="stSidebar"] input {
    font-size: 12.5px !important;
    padding: 4px 8px !important;
    height: 34px !important;
    color: #0f172a !important;
}

/* Tema Azul para Controles: Radio, Toggles, Checkboxes */
div[role="radiogroup"] label div[role="radio"][aria-checked="true"],
div[role="radiogroup"] label div:has(input:checked),
div[data-testid="stRadio"] label [aria-checked="true"] > div,
div[data-testid="stRadio"] [aria-checked="true"] div {
    border-color: #005cb2 !important;
}
div[role="radiogroup"] label div[role="radio"][aria-checked="true"]::after,
div[data-testid="stRadio"] [aria-checked="true"] span,
div[data-testid="stRadio"] [aria-checked="true"] svg circle {
    background-color: #005cb2 !important;
    fill: #005cb2 !important;
}

div[data-testid="stToggle"] label div[aria-checked="true"],
div[data-testid="stToggle"] [aria-checked="true"] {
    background-color: #005cb2 !important;
}

div[data-testid="stCheckbox"] label div[aria-checked="true"],
div[data-testid="stCheckbox"] [aria-checked="true"] {
    background-color: #005cb2 !important;
    border-color: #005cb2 !important;
}

div[data-testid="stNumberInputStepDown"],
div[data-testid="stNumberInputStepUp"],
div[data-testid="stNumberInput"] button {
    width: 1.5rem !important;
    height: 1.5rem !important;
    min-width: 1.5rem !important;
    padding: 0 !important;
    display: inline-flex !important;
}
div[data-testid="stNumberInputStepDown"] svg,
div[data-testid="stNumberInputStepUp"] svg,
div[data-testid="stNumberInput"] button svg {
    width: 12px !important;
    height: 12px !important;
}

/* Botones de incremento y decremento (+ y -) de stNumberInput */
div[data-testid="stNumberInput"] button:hover,
div[data-testid="stNumberInput"] button:focus,
div[data-testid="stNumberInput"] button:active,
div[data-testid="stNumberInputStepDown"]:hover,
div[data-testid="stNumberInputStepDown"]:focus,
div[data-testid="stNumberInputStepDown"]:active,
div[data-testid="stNumberInputStepUp"]:hover,
div[data-testid="stNumberInputStepUp"]:focus,
div[data-testid="stNumberInputStepUp"]:active {
```

```
    background-color: #005cb2 !important;
    color: #ffffff !important;
    border-color: #005cb2 !important;
}
div[data-testid="stNumberInput"] button:hover svg,
div[data-testid="stNumberInput"] button:focus svg,
div[data-testid="stNumberInput"] button:active svg,
div[data-testid="stNumberInputStepDown"]:hover svg,
div[data-testid="stNumberInputStepDown"]:focus svg,
div[data-testid="stNumberInputStepDown"]:active svg,
div[data-testid="stNumberInputStepUp"]:hover svg,
div[data-testid="stNumberInputStepUp"]:focus svg,
div[data-testid="stNumberInputStepUp"]:active svg {
    fill: #ffffff !important;
    stroke: #ffffff !important;
}

/* ========================================================= */
/* Submenús / Expanders con apariencia de tarjetas redondeadas */
/* ========================================================= */
div[data-testid="stExpander"],
.stExpander {
    border: none !important;
    background: transparent !important;
    box-shadow: none !important;
    margin-bottom: 7px !important;
    margin-right: 15px !important;
}

section[data-testid="stSidebar"] div[data-testid="stExpander"] details,
div[data-testid="stExpander"] details {
    border: 1.5px solid #cbd5e1 !important;
    border-radius: 8px !important;
    background-color: #f8fafc !important;
    overflow: hidden !important;
    transition: all 0.2s ease !important;
    box-shadow: 0 1px 2px rgba(0, 0, 0, 0.03) !important;
}

section[data-testid="stSidebar"] div[data-testid="stExpander"] details:hover,
div[data-testid="stExpander"] details:hover {
    border-color: #94a3b8 !important;
    background-color: #f1f5f9 !important;
}

/* Submenú activo / desplegado (Borde azul continuo) */
section[data-testid="stSidebar"] div[data-testid="stExpander"] details[open],
div[data-testid="stExpander"] details[open] {
    border: 2px solid #5b92cb !important;
    background-color: #f8fafc !important;
    box-shadow: 0 2px 8px rgba(0, 92, 178, 0.12) !important;
}

/* Encabezado del Submenú */
section[data-testid="stSidebar"] div[data-testid="stExpander"] details summary,
div[data-testid="stExpander"] details summary {
    padding: 7px 12px !important;
    background-color: #f8fafc !important;
    color: #1e293b !important;
    font-size: 13px !important;
    font-weight: 500 !important;
    border-radius: 6px !important;
    display: flex !important;
```

```
    align-items: center !important;
    cursor: pointer !important;
}

section[data-testid="stSidebar"] div[data-testid="stExpander"] details summary:hover,
div[data-testid="stExpander"] details summary:hover {
    background-color: #f1f5f9 !important;
}

/* Encabezado cuando está abierto (fondo azul tenue y borde inferior divisorio) */
section[data-testid="stSidebar"] div[data-testid="stExpander"] details[open] summary,
div[data-testid="stExpander"] details[open] summary {
    background-color: #e8f1fa !important;
    border-bottom: 1px solid #cbd5e1 !important;
    border-radius: 6px 6px 0 0 !important;
    font-weight: 600 !important;
    color: #0f172a !important;
}

section[data-testid="stSidebar"] div[data-testid="stExpander"] details summary p,
div[data-testid="stExpander"] details summary p {
    font-size: 13px !important;
    font-weight: inherit !important;
    color: inherit !important;
    margin: 0 !important;
}

section[data-testid="stSidebar"] div[data-testid="stExpander"] details summary svg,
div[data-testid="stExpander"] details summary svg {
    color: #475569 !important;
    width: 14px !important;
    height: 14px !important;
}

/* Contenido dentro del Submenú */
section[data-testid="stSidebar"] div[data-testid="stExpander"] [data-testid="stExpanderDetails"],
div[data-testid="stExpander"] [data-testid="stExpanderDetails"] {
    padding: 8px 10px 10px 10px !important;
    background-color: #f8fafc !important;
    border-radius: 0 0 6px 6px !important;
}

/* Ajuste de fuente para los botones */
button, button p, button div, button span {
    font-weight: 600 !important;
    font-size: 13.5px !important;
}

/* Botones principales (Azul Sólido) */
section[data-testid="stSidebar"] button[kind="primary"] {
    background: #005cb2 !important;
    color: white !important;
    font-weight: 600 !important;
    font-size: 13.5px !important;
    border-radius: 4px !important;
    width: 100% !important;
    margin: 0 auto !important;
    display: block !important;
    border: none !important;
    text-align: center !important;
    justify-content: center !important;
    box-shadow: 0px 1px 3px rgba(0, 0, 0, 0.1) !important;
    transition: all 0.2s ease-in-out !important;
```

```
    padding-top: 0.5rem !important;
    padding-bottom: 0.5rem !important;
}
section[data-testid="stSidebar"] button[kind="primary"]:hover {
    background: #004ba0 !important;
    box-shadow: 0px 3px 6px rgba(0, 0, 0, 0.15) !important;
    transform: translateY(-1px) !important;
}
section[data-testid="stSidebar"] button[kind="primary"]:active {
    transform: translateY(1px) !important;
    box-shadow: 0px 1px 2px rgba(0, 0, 0, 0.1) !important;
}

/* Botón de retroceso (Flecha gris) */
section[data-testid="stSidebar"] button[kind="secondary"] {
    background-color: #f8f9fa !important;
    color: #444 !important;
    border: 1px solid #ddd !important;
    padding: 3px 10px !important;
    width: auto !important;
    font-size: 13px !important;
    font-weight: 600 !important;
    box-shadow: 0px 1px 2px rgba(0,0,0,0.05) !important;
    border-radius: 4px !important;
    margin-bottom: 4px !important;
    margin-top: 4px !important;
    position: relative !important;
    z-index: 10 !important;
}
section[data-testid="stSidebar"] button[kind="secondary"]:hover {
    background-color: #e9ecef !important;
    color: #111 !important;
}

/* Botón de Guardar/Acciones en Panel Derecho (Rojo Sólido) */
div[data-testid="stMainBlockContainer"] button[kind="primary"] {
    background: #ba281b !important;
    color: white !important;
    font-weight: 600 !important;
    border-radius: 4px !important;
    border: none !important;
    box-shadow: 0px 1px 3px rgba(0, 0, 0, 0.1) !important;
    transition: all 0.2s ease-in-out !important;
    padding-top: 0.5rem !important;
    padding-bottom: 0.5rem !important;
}
div[data-testid="stMainBlockContainer"] button[kind="primary"]:hover {
    background: #9d2116 !important;
    box-shadow: 0px 3px 6px rgba(0, 0, 0, 0.15) !important;
    transform: translateY(-1px) !important;
}
div[data-testid="stMainBlockContainer"] button[kind="primary"]:active {
    transform: translateY(1px) !important;
    box-shadow: 0px 1px 2px rgba(0, 0, 0, 0.1) !important;
}

/* Marcador para botones grises (Diseño y Catálogo) */
div.element-container:has(div.grey-btn-marker), div[data-testid="stElementContainer"]:has(div.grey-btn-marker) {
    display: none !important;
    margin: 0 !important;
    height: 0 !important;
}
```

```
/* Botones de Diseño y Catálogo (Gris siempre) */
div.element-container:has(div.grey-btn-marker) + div.element-container button,
div[data-testid="stElementContainer"]:has(div.grey-btn-marker) + div[data-testid="stElementContainer"] button,
section[data-testid="stSidebar"] div.element-container:has(div.grey-btn-marker) + div.element-container button,
section[data-testid="stSidebar"] div[data-testid="stElementContainer"]:has(div.grey-btn-marker) + div[data-testid=
    background: #6c757d !important;
    color: white !important;
    font-weight: 600 !important;
    border-radius: 4px !important;
    border: none !important;
    width: 100% !important;
    margin: 0 auto !important;
    display: block !important;
```

Muestra de Código - Motor Lógico (Últimas páginas)

```
import math

class StaadModel:
    def __init__(self, Larm_m, Ljoist_m, nBay, nSpan, HArmad_m, HLibre_m, numSubdiv, roof_type="Plana", slope_pct=0.0,
        self.Larm_m = float(Larm_m)
        self.Ljoist_m = float(Ljoist_m)
        self.nBay = int(nBay)
        self.nSpan = int(nSpan)
        self.HArmad_m = float(HArmad_m)
        self.HLibre_m = float(HLibre_m)
        self.numSubdiv = int(numSubdiv)

        self.Larm_list = [self.Larm_m] * self.nSpan
        self.subdiv_list = [self.numSubdiv] * self.nSpan

        if kwargs.get('dif_lengths', False) and self.nSpan >= 2:
            self.Larm_list[0] = float(kwargs.get('larm_ext1', self.Larm_m))
            self.subdiv_list[0] = int(kwargs.get('subdiv_ext1', self.numSubdiv))
            if self.nSpan > 2:
                self.Larm_list[-1] = float(kwargs.get('larm_ext2', self.Larm_m))
                self.subdiv_list[-1] = int(kwargs.get('subdiv_ext2', self.numSubdiv))
            elif self.nSpan == 2:
                self.Larm_list[1] = float(kwargs.get('larm_ext2', self.Larm_m))
                self.subdiv_list[1] = int(kwargs.get('subdiv_ext2', self.numSubdiv))

        for s in self.subdiv_list:
            if s < 2 or s % 2 != 0:
                raise ValueError("Las subdivisiones deben ser un número par mayor o igual a 2.")

        self.roof_type = roof_type
        self.slope_pct = float(slope_pct)
        self.support_type = support_type
        self.resorte_val = resorte_val
        self.wall_type = wall_type

        self.ext_bays_active = kwargs.get('ext_bays_active', False)
        self.ext_bl = kwargs.get('ext_bl', False)
        self.ext_br = kwargs.get('ext_br', False)
        self.ext_tl = kwargs.get('ext_tl', False)
        self.ext_tr = kwargs.get('ext_tr', False)
        self.ext_height_mode = kwargs.get('ext_height_mode', "Primer Entre-Eje completo")
        self.ext_depth = int(kwargs.get('ext_depth', 1))
        self.ext_larm = float(kwargs.get('ext_larm', 15.24)) * 100.0
        self.ext_subdiv = int(kwargs.get('ext_subdiv', 10))

        self.sec_trusses_j = sec_trusses_j if sec_trusses_j else []
        if self.sec_trusses_j:
            import math
            min_n = self.Ljoist_m / 1.60
            n = math.ceil(min_n)
            if n % 2 != 0: n += 1
            self.numSubdiv_sec = int(max(2, n))
        else:
            self.numSubdiv_sec = 0

        self.c_muerta = float(c_muerta)
        self.c_viva = float(c_viva)
        self.c_granizo = float(c_granizo)
```

```
self.v_barlovento = float(v_barlovento)
self.v_sotavento = float(v_sotavento)
self.v_laterales = float(v_laterales)
self.v_succion = float(v_succion)

self.coef_sismico = float(coef_sismico)
self.ductilidad = float(ductilidad)
self.cm_sismo = float(cm_sismo) if cm_sismo is not None else self.c_muerta
self.cv_sismo = float(cv_sismo) if cv_sismo is not None else (self.c_viva * 0.25)
self.seismic_v = self.coef_sismico / self.ductilidad

self.add_wind_cols = bool(add_wind_cols)
self.max_wind_dist_m = float(max_wind_dist_m)

import math
self.wind_col_N = 1
if self.add_wind_cols and self.Ljoist_m > self.max_wind_dist_m:
    self.wind_col_N = math.ceil(self.Ljoist_m / self.max_wind_dist_m)

self.wind_col_Z_M_list = []
for l in self.Larm_list:
    if self.add_wind_cols and l > self.max_wind_dist_m:
        self.wind_col_Z_M_list.append(math.ceil(l / self.max_wind_dist_m))
    else:
        self.wind_col_Z_M_list.append(1)

self.scale = 100.0
self.Lx_list = [l * self.scale for l in self.Larm_list]
self.Lz = self.Ljoist_m * self.scale
self.HtArmad = self.HArmad_m * self.scale
self.HtLibre = self.HLibre_m * self.scale
self.total_width = sum(self.Lx_list)
self.Lx = self.Lx_list[0] # Legacy fallback if needed outside generate

self.y_shift_ext = 0.0
if self.ext_bays_active and self.ext_height_mode == "Altura mínima en esquinas":
    slope = self.slope_pct / 100.0
    W_left = self.ext_larm if (self.ext_bl or self.ext_tl) else 0.0
    W_right = self.ext_larm if (self.ext_br or self.ext_tr) else 0.0

    if self.roof_type == "1 Agua":
        self.y_shift_ext = W_left * slope
    elif self.roof_type == "2 Aguas":
        self.y_shift_ext = max(W_left * slope, W_right * slope)

self.col_trib_z = {}
self.col_trib_x = {}
self.node_seismic_loads = {}
self.node_seismic_areas = {}
self.seismic_load_groups = {}
self.nodes = {}  # node_id: (x, y, z)
self.members = {}  # member_id: (node1_id, node2_id)
self.groups = {
    '_COL': [],
    '_COLVIENTO': [],
    '_CS': [],
    '_CI': [],
    '_CS_PERIM': [],
    '_CI_PERIM': [],
    '_MONTANTES': [],
    '_DIAG': [],
    '_MONTANTES_PERIM': [],
    '_DIAG_PERIM': [],
```

```
    '_LARGUEROS': [],
    '_CVH': [],
    '_JOISTGRAL': [],
    '_JPUNTAL': [],
    '_CS_SEC': [],
    '_CI_SEC': [],
    '_MONTANTES_SEC': [],
    '_DIAG_SEC': [],
    '_ARMADURA_MD': []
}
# Agrupación de largueros según su separación transversal (s = Larm / subdiv)
self.span_larguero_spacing = [
    round(self.Larm_list[j] / self.subdiv_list[j], 4)
    for j in range(self.nSpan)
]

unique_spacings = []
for s in self.span_larguero_spacing:
    if s not in unique_spacings:
        unique_spacings.append(s)

if self.ext_bays_active:
    s_ext = round((self.ext_larm / 100.0) / self.ext_subdiv, 4)
    if s_ext not in unique_spacings:
        unique_spacings.append(s_ext)

self.largueros_group_info = {}
self.span_to_larguero_group = {}

if len(unique_spacings) <= 1:
    main_grp = '_LARGUEROS'
    spacing_val = unique_spacings[0] if unique_spacings else round(self.Larm_m / self.numSubdiv, 4)
    self.largueros_group_info[main_grp] = {
        'spacing': spacing_val,
        'spans': list(range(self.nSpan))
    }
    for j in range(self.nSpan + 1):
        self.span_to_larguero_group[j] = main_grp
    self.ext_larguero_group = main_grp
else:
    spacing_to_grp = {}
    for idx, s in enumerate(unique_spacings, 1):
        grp_name = f"_LARGUEROS_{idx}"
        spacing_to_grp[s] = grp_name
        self.largueros_group_info[grp_name] = {
            'spacing': s,
            'spans': [j for j in range(self.nSpan) if self.span_larguero_spacing[j] == s]
        }
    for j in range(self.nSpan):
        self.span_to_larguero_group[j] = spacing_to_grp[self.span_larguero_spacing[j]]
    # Para el eje derecho extremo j = nSpan, asociar a la separación del último claro
    self.span_to_larguero_group[self.nSpan] = spacing_to_grp[self.span_larguero_spacing[-1]]
    if self.ext_bays_active:
        s_ext = round((self.ext_larm / 100.0) / self.ext_subdiv, 4)
        self.ext_larguero_group = spacing_to_grp[s_ext]
    else:
        self.ext_larguero_group = self.span_to_larguero_group[0]

for grp in self.largueros_group_info.keys():
    self.groups[grp] = []
for j in range(self.nSpan + 1):
    self.groups[f"_CS_SEC_{j}"] = []
```

```
self.wind_cols = {
    'X0_CORNER': [], 'X0_INNER': [],
    'X0_WINDCOL': [],
    'XMAX_CORNER': [], 'XMAX_INNER': [],
    'XMAX_WINDCOL': [],
    'Z0_CORNER': [], 'Z0_INNER': [],
    'Z0_WINDCOL': [],
    'ZMAX_CORNER': [], 'ZMAX_INNER': [],
    'ZMAX_WINDCOL': []
}

# Inicialización de áreas tributarias dinámicas de sismo
pass

self.rel_csci_start = []
self.rel_csci_end = []
self.rel_horiz_start = []
self.rel_horiz_end = []
self.rel_larg_start = []
self.rel_larg_end = []
self.rel_cs_sec_start = []
self.rel_cs_sec_end = []
self.rel_ci_sec_start = []
self.rel_ci_sec_end = []
self.rel_tiltup_start = []
self.rel_tiltup_end = []
self.plate_releases_j1_j4 = []
self.plate_releases_j2_j3 = []
self.release_end_fx = []
self.horizontal_truss_members = []
self.split_purlin_first = []
self.split_purlin_last = []
self.split_purlin_middle = []

# Marco Doble / Juntas Constructivas
self.junta_constructiva_tipo = kwargs.get('junta_constructiva_tipo', 'Ninguna')
self.junta_max_dist_ft = float(kwargs.get('junta_max_dist_ft', 600.0))
self.junta_max_dist_m = self.junta_max_dist_ft * 0.3048

total_length_m = self.nBay * self.Ljoist_m
self.has_marco_doble = (self.junta_constructiva_tipo == "Marco doble") and (total_length_m > self.junta_max_di
self.has_junta_cubierta = (self.junta_constructiva_tipo == "En Cubierta") and (total_length_m > self.junta_max
self.joint_axes = []
if (self.has_marco_doble or self.has_junta_cubierta) and self.nBay >= 2:
    num_segments = math.ceil((total_length_m - 0.05) / self.junta_max_dist_m)
    num_joints = num_segments - 1
    for k in range(1, num_joints + 1):
        target_i = int(round(k * self.nBay / (num_joints + 1)))
        target_i = max(1, min(self.nBay - 1, target_i))
        if target_i not in self.joint_axes:
            self.joint_axes.append(target_i)
    self.joint_axes.sort()

self.roof_joint_bays = {} # bay_idx -> 'START' or 'END' (extremo que conecta al marco divisor)
if self.has_junta_cubierta:
    for j_ax in self.joint_axes:
        if j_ax <= self.nBay / 2.0:
            self.roof_joint_bays[j_ax] = 'START'
        else:
            self.roof_joint_bays[j_ax - 1] = 'END'

self.rel_roof_joint_start = []
self.rel_roof_joint_end = []
```

```
    self.roof_joint_purlins_start = set()
    self.roof_joint_purlins_end = set()

    # Construir marcos a lo largo de Z
    self.frames = []
    f_idx = 0
    for i in range(self.nBay + 1):
        if self.has_marco_doble and i in self.joint_axes:
            zA = i * self.Lz - 10.0
            self.frames.append({
                'idx': f_idx,
                'z': zA,
                'orig_i': i,
                'is_md': True,
                'md_side': 'A',
                'connect_to_next': False
            })
            f_idx += 1
            zB = i * self.Lz + 10.0
            self.frames.append({
                'idx': f_idx,
                'z': zB,
                'orig_i': i,
                'is_md': True,
                'md_side': 'B',
                'connect_to_next': True
            })
            f_idx += 1
        else:
            self.frames.append({
                'idx': f_idx,
                'z': i * self.Lz,
                'orig_i': i,
                'is_md': False,
                'md_side': None,
                'connect_to_next': True
            })
            f_idx += 1
    self.num_frames = len(self.frames)
    self.dummy_md_members = []

    self.col_heights = {}
    self.ci_int = []
    self.ci_ly_groups = {}

    self.base_nodes = [] # Nodos en la base para asignar SUPPORTS FIXED
    self.wind_base_nodes = []
    self.plate_groups = {'X0': [], 'XMAX': [], 'Z0': [], 'ZMAX': []}
    self.ext_corners = []
    self.ext_outer_roof_nodes = {}

    self.node_id = 0
    self.mem_id = 0

    self.plates = {} # mem_id: (n1, n2, n3, n4)

    self.generate()

def add_node(self, x, y, z, is_wind_base=False):
    self.node_id += 1
    self.nodes[self.node_id] = (x, y, z)
    if y == 0:
        if is_wind_base:
```

```
            self.wind_base_nodes.append(self.node_id)
        else:
            self.base_nodes.append(self.node_id)

    if self.wall_type == "Tilt-Up":
        tiltup_base_nodes = set()
        for pid, (n1, n2, n3, n4) in self.plates.items():
            if self.nodes[n1][1] == 0.0: tiltup_base_nodes.add(n1)
            if self.nodes[n2][1] == 0.0: tiltup_base_nodes.add(n2)
            if self.nodes[n3][1] == 0.0: tiltup_base_nodes.add(n3)
            if self.nodes[n4][1] == 0.0: tiltup_base_nodes.add(n4)
        for nid in tiltup_base_nodes:
            if nid in self.base_nodes:
                self.base_nodes.remove(nid)
            if nid not in self.wind_base_nodes:
                self.wind_base_nodes.append(nid)

    return self.node_id

def add_plate(self, n1, n2, n3, n4):
    self.mem_id += 1
    self.plates[self.mem_id] = (n1, n2, n3, n4)

    if self.wall_type == "Tilt-Up":
        tiltup_base_nodes = set()
        for pid, (n1, n2, n3, n4) in self.plates.items():
            if self.nodes[n1][1] == 0.0: tiltup_base_nodes.add(n1)
            if self.nodes[n2][1] == 0.0: tiltup_base_nodes.add(n2)
            if self.nodes[n3][1] == 0.0: tiltup_base_nodes.add(n3)
            if self.nodes[n4][1] == 0.0: tiltup_base_nodes.add(n4)
        for nid in tiltup_base_nodes:
            if nid in self.base_nodes:
                self.base_nodes.remove(nid)
            if nid not in self.wind_base_nodes:
                self.wind_base_nodes.append(nid)

    return self.mem_id

def add_member(self, n1, n2, group):
    self.mem_id += 1
    self.members[self.mem_id] = (n1, n2)
    if group in self.groups:
        self.groups[group].append(self.mem_id)

    if self.wall_type == "Tilt-Up":
        tiltup_base_nodes = set()
        for pid, (n1, n2, n3, n4) in self.plates.items():
            if self.nodes[n1][1] == 0.0: tiltup_base_nodes.add(n1)
            if self.nodes[n2][1] == 0.0: tiltup_base_nodes.add(n2)
            if self.nodes[n3][1] == 0.0: tiltup_base_nodes.add(n3)
            if self.nodes[n4][1] == 0.0: tiltup_base_nodes.add(n4)
        for nid in tiltup_base_nodes:
            if nid in self.base_nodes:
                self.base_nodes.remove(nid)
            if nid not in self.wind_base_nodes:
                self.wind_base_nodes.append(nid)

    return self.mem_id

def get_upper_node(self, nL3, upperSub, i, j, k):
    if k == 0:
        return nL3[(i, j)]
    elif k == self.subdiv_list[j]:
```

```
        return nL3[(i, j + 1)]
    else:
        return upperSub[(i, j, k)]

def add_ci_ly(self, mem_id, L0_local):
    ly_val = math.ceil(3 * L0_local)
    if ly_val not in self.ci_ly_groups:
        self.ci_ly_groups[ly_val] = []
    self.ci_ly_groups[ly_val].append(mem_id)

def get_x(self, j):
    return sum(self.Lx_list[:j])

def get_y_offset(self, global_x):
    slope = self.slope_pct / 100.0
    if self.roof_type == "1 Agua":
        return global_x * slope + self.y_shift_ext
    elif self.roof_type == "2 Aguas":
        return min(global_x, self.total_width - global_x) * slope + self.y_shift_ext

    if self.wall_type == "Tilt-Up":
        tiltup_base_nodes = set()
        for pid, (n1, n2, n3, n4) in self.plates.items():
            if self.nodes[n1][1] == 0.0: tiltup_base_nodes.add(n1)
            if self.nodes[n2][1] == 0.0: tiltup_base_nodes.add(n2)
            if self.nodes[n3][1] == 0.0: tiltup_base_nodes.add(n3)
            if self.nodes[n4][1] == 0.0: tiltup_base_nodes.add(n4)
        for nid in tiltup_base_nodes:
            if nid in self.base_nodes:
                self.base_nodes.remove(nid)
            if nid not in self.wind_base_nodes:
                self.wind_base_nodes.append(nid)

    return self.y_shift_ext



def get_or_create_node(self, x, y, z):
    tol = 1e-4
    for nid, (nx, ny, nz) in self.nodes.items():
        if abs(nx - x) < tol and abs(ny - y) < tol and abs(nz - z) < tol:
            return nid

    if self.wall_type == "Tilt-Up":
        tiltup_base_nodes = set()
        for pid, (n1, n2, n3, n4) in self.plates.items():
            if self.nodes[n1][1] == 0.0: tiltup_base_nodes.add(n1)
            if self.nodes[n2][1] == 0.0: tiltup_base_nodes.add(n2)
            if self.nodes[n3][1] == 0.0: tiltup_base_nodes.add(n3)
            if self.nodes[n4][1] == 0.0: tiltup_base_nodes.add(n4)
        for nid in tiltup_base_nodes:
            if nid in self.base_nodes:
                self.base_nodes.remove(nid)
            if nid not in self.wind_base_nodes:
                self.wind_base_nodes.append(nid)

    return self.add_node(x, y, z)

def _add_sec_truss_segment(self, j_span, top_start, top_end, bot_start, bot_end, base_z, L_segment, start_release,
    min_n = L_segment / (1.60 * self.scale)
    n = math.ceil(min_n)
    if n % 2 != 0: n += 1
    num_subdiv = int(max(2, n))
```

```
L0_sec = L_segment / num_subdiv

sec_upper = {0: top_start}
for k in range(1, num_subdiv):
    zn = base_z + k * L0_sec
    xn = self.get_x(j_span)
    yn_roof = self.HtLibre + self.HtArmad + self.get_y_offset(xn)
    sec_upper[k] = self.get_or_create_node(xn, yn_roof, zn)
sec_upper[num_subdiv] = top_end

sec_lower = {}
halfSub = num_subdiv // 2
for t in range(1, halfSub + 1):
```